

CSS User-Select Property

Complete Guide: Easy English Explanation

Contents

1	Introduction	2
1.1	What is User-Select?	2
1.2	Why Use User-Select?	2
1.3	Use With Caution	2
2	Core Concept: User-Select Values	4
2.1	Value 1: <code>auto</code> (Default)	4
2.1.1	What It Does	4
2.1.2	Code Example	4
2.1.3	When to Use <code>auto</code>	4
2.2	Value 2: <code>none</code>	4
2.2.1	What It Does	4
2.2.2	Code Example	4
2.2.3	Visual Result	5
2.2.4	When to Use <code>none</code>	5
2.3	Value 3: <code>text</code>	5
2.3.1	What It Does	5
2.3.2	Code Example	5
2.3.3	When to Use <code>text</code>	5
2.4	Value 4: <code>all</code>	5
2.4.1	What It Does	5
2.4.2	Code Example	6
2.4.3	Visual Result	6
2.4.4	When to Use <code>all</code>	6
2.5	Value 5: <code>contain</code>	6
2.5.1	What It Does	6
2.5.2	Code Example	6
2.5.3	When to Use <code>contain</code>	6
3	Side-by-Side Comparison	7
3.1	All Values Compared	7
3.1.1	Scenario: User drags mouse to select text	7
3.2	Comparison Table	8
4	Vendor Prefixes	9
4.1	Browser Compatibility	9

5	Practical Examples	10
5.1	Example 1: Non-Selectable Button	10
5.2	Example 2: Protected Navigation Menu	10
5.3	Example 3: Copyable Code Block	10
5.4	Example 4: Mixed Content - Selectable Text, Non-Selectable UI	11
5.5	Example 5: Drag-and-Drop Interface	11
5.6	Example 6: Icon with Label	12
5.7	Example 7: Card Component	12
5.8	Example 8: Tooltip and Help Text	13
5.9	Example 9: Disable Selection Globally (Rare)	13
5.10	Example 10: Form Labels	13
6	Common Mistakes	15
6.1	Mistake 1: Using <code>none</code> on Article Text	15
6.2	Mistake 2: Thinking user-select Prevents Copying	15
6.3	Mistake 3: Over-Protecting UI	16
6.4	Mistake 4: Forgetting Vendor Prefixes	16
6.5	Mistake 5: Using <code>none</code> on Accessible Content	16
7	Accessibility Considerations	17
7.1	Screen Readers are Unaffected	17
7.2	Best Practices for Accessibility	17
8	Browser Support	18
8.1	Current Support	18
8.2	Always Use Vendor Prefixes	18
9	Related CSS Properties	19
9.1	Complementary Properties	19
9.1.1	cursor	19
9.1.2	pointer-events	19
9.1.3	user-drag	19
10	Quick Reference	20
10.1	User-Select Values Summary	20
10.2	Complete CSS Template	20
10.3	Decision Flowchart	21
10.4	Common Patterns	21
10.5	Pro Tips	21
10.6	When NOT to Use user-select: none	22
10.7	When to Use user-select: none	22

Chapter 1

Introduction

1.1 What is User-Select?

The `user-select` property controls whether users can select (highlight/copy) text on a webpage with their mouse or keyboard. It determines what content can be highlighted, copied, and pasted.

Think of user-select like document protection:

- **Default:** Users can select and copy everything (like an editable document)
- **With user-select control:** Some content can be locked (can't select/copy)

User-select gives you control over what content users can copy from your page!

1.2 Why Use User-Select?

User-select is useful for:

- Protecting UI buttons and labels from accidental selection
- Preventing copy of decorative elements and icons
- Improving user experience by reducing unwanted highlights
- Protecting intellectual property and brand assets
- Making interfaces feel more polished and intentional
- Preventing selection interference with drag-and-drop features
- Controlling what content appears in copy-paste actions
- Protecting sensitive design elements

1.3 Use With Caution

Important Ethical Considerations:

- Don't disable selection for article text (readers want to copy text)
- Don't disable for educational or blog content

- Don't prevent copy to restrict access (ineffective, users can still copy)
- Use sparingly for UI elements only, not content
- Respect user accessibility needs
- Consider that copy protection can frustrate users

Chapter 2

Core Concept: User-Select Values

2.1 Value 1: **auto** (Default)

2.1.1 What It Does

The **auto** value uses the browser's default selection behavior. Text can be selected and copied normally. This is the standard behavior for most elements.

2.1.2 Code Example

```
p {  
  user-select: auto; /* or just omit - default */  
  /* Text can be selected normally */  
}
```

2.1.3 When to Use **auto**

- For body text and article content
- For any content users should be able to copy
- Default for most elements

2.2 Value 2: **none**

2.2.1 What It Does

The **none** value prevents users from selecting text on the element. Text cannot be highlighted, even if they triple-click or drag to select. However, the text still appears on the page normally and can be accessed by screen readers.

2.2.2 Code Example

```
button {  
  user-select: none;  
  /* Button text cannot be selected or highlighted */  
}
```

2.2.3 Visual Result

User clicks and drags across text, but nothing gets highlighted. Text appears unselectable.

2.2.4 When to Use **none**

- Button labels
- Navigation menu items
- UI labels and controls
- Icon labels
- Decorative text elements
- Drag-and-drop handles
- NOT for article content (bad for UX)

2.3 Value 3: **text**

2.3.1 What It Does

The **text** value allows users to select text. This is similar to **auto** but more explicitly states that selection is allowed. Selection works normally.

2.3.2 Code Example

```
.article {  
  user-select: text;  
  /* Text can be selected normally */  
}
```

2.3.3 When to Use **text**

- When you explicitly want to allow selection
- To override inherited **none** from parent
- For content that should be copyable
- Less common than **auto**

2.4 Value 4: **all**

2.4.1 What It Does

The **all** value makes the entire element's text selectable as a single block. Double-clicking selects all text in the element, not just a single word. Selection works differently than normal word-by-word selection.

2.4.2 Code Example

```
.code-block {  
  user-select: all;  
  /* Double-click selects entire block */  
}
```

2.4.3 Visual Result

When user clicks anywhere in the element, the entire text block becomes selected. Useful for code snippets where users typically want everything.

2.4.4 When to Use `all`

- Code snippets and code blocks
- Command-line examples
- Share URLs or API endpoints
- Content meant to be copied entirely
- Less common, niche use

2.5 Value 5: `contain`

2.5.1 What It Does

The `contain` value allows selection within the element, but prevents selection from crossing element boundaries. If a user tries to drag-select across multiple elements with `contain`, the selection stops at the boundary.

2.5.2 Code Example

```
.contained-item {  
  user-select: contain;  
  /* Selection works inside, stops at borders */  
}
```

2.5.3 When to Use `contain`

- Isolating selectable regions
- List items that shouldn't select together
- Card components with independent content
- Rarely used in practice
- Limited browser support

Chapter 3

Side-by-Side Comparison

3.1 All Values Compared

3.1.1 Scenario: User drags mouse to select text

auto (default)

```
p {  
  user-select: auto;  
}  
  
/* Result: Text highlights normally, can be copied */
```

none

```
button {  
  user-select: none;  
}  
  
/* Result: Nothing happens, no highlight, can't select */
```

text

```
.content {  
  user-select: text;  
}  
  
/* Result: Text highlights normally, can be copied */  
/* Same as auto, just explicit */
```

all

```
.code {  
  user-select: all;  
}  
  
/* Result: Any click selects entire block at once */
```

3.2 Comparison Table

Value	Selectable	Copy Works	Use Case
auto	YES	YES	Content
none	NO	NO	UI elements
text	YES	YES	Content (explicit)
all	YES (all at once)	YES	Code blocks
contain	YES (bounded)	YES	List items

Chapter 4

Vendor Prefixes

4.1 Browser Compatibility

For maximum compatibility with older browsers:

```
.element {  
  -webkit-user-select: none;  
  -moz-user-select: none;  
  -ms-user-select: none;  
  user-select: none;  
}
```

Important vendor prefixes:

- **-webkit-** for Chrome, Safari, Edge (Chromium)
- **-moz-** for Firefox
- **-ms-** for older Edge and IE

Chapter 5

Practical Examples

5.1 Example 1: Non-Selectable Button

```
button {  
  user-select: none;  
  /* Button text won't highlight when clicking */  
}  
  
button:active {  
  opacity: 0.8;  
  /* Still shows active state */  
}
```

Result: Professional button that doesn't get highlighted during use.

5.2 Example 2: Protected Navigation Menu

```
nav {  
  user-select: none;  
  /* All navigation items non-selectable */  
}  
  
nav a {  
  user-select: none;  
  display: block;  
  padding: 12px 16px;  
  transition: background 0.3s;  
}  
  
nav a:hover {  
  background: #f0f0f0;  
  /* Hover effect still works */  
}
```

Result: Clean navigation that doesn't highlight unwanted when clicking.

5.3 Example 3: Copyable Code Block

```
.code-block {
  background: #f5f5f5;
  padding: 16px;
  border-radius: 4px;
  font-family: monospace;
  user-select: all;
  /* Clicking anywhere selects entire code */
}

.code-block:hover {
  background: #eeeeee;
}
```

Result: Users can easily copy entire code snippet by clicking.

5.4 Example 4: Mixed Content - Selectable Text, Non-Selectable UI

```
/* Article text is selectable */
article p {
  user-select: auto;
}

/* But buttons and labels are not */
.button,
.label,
.tag {
  user-select: none;
}

/* Code within article is special */
code {
  user-select: all;
}
```

Result: Article text can be copied, but UI elements are protected.

5.5 Example 5: Drag-and-Drop Interface

```
.draggable-item {
  user-select: none;
  /* Prevent selection during drag */
  cursor: grab;
  padding: 12px;
  background: #3498db;
  color: white;
  border-radius: 4px;
  margin: 8px 0;
}
```

```
.draggable-item:active {  
  cursor: grabbing;  
}  
  
.draggable-item.dragging {  
  opacity: 0.7;  
  transform: rotate(-5deg);  
}
```

Result: Items can be dragged without accidental text selection.

5.6 Example 6: Icon with Label

```
.icon-with-label {  
  display: flex;  
  align-items: center;  
  gap: 8px;  
  user-select: none;  
  /* Icon and label won't highlight on click */  
}  
  
.icon {  
  font-size: 24px;  
}  
  
.label {  
  font-weight: 600;  
}
```

Result: Icon and label stay clean looking without selection highlight.

5.7 Example 7: Card Component

```
.card {  
  border: 1px solid #e0e0e0;  
  border-radius: 8px;  
  padding: 16px;  
}  
  
.card-header {  
  user-select: none;  
  font-size: 18px;  
  font-weight: bold;  
  margin-bottom: 12px;  
  /* Header title won't be selected */  
}  
  
.card-body {  
  user-select: auto;  
  color: #666;  
  line-height: 1.6;  
  /* Card content is selectable */  
}
```

```
}
```

Result: Card headers look clean, content is copyable.

5.8 Example 8: Tooltip and Help Text

```
.tooltip {  
  position: absolute;  
  background: #333;  
  color: white;  
  padding: 8px 12px;  
  border-radius: 4px;  
  font-size: 12px;  
  user-select: none;  
  /* Tooltip won't interfere with selection */  
  pointer-events: none;  
  /* Doesn't block clicks */  
  z-index: 1000;  
}
```

Result: Tooltips don't interrupt user workflow.

5.9 Example 9: Disable Selection Globally (Rare)

```
body {  
  user-select: none;  
  /* Everything non-selectable by default */  
}  
  
article,  
.content,  
.article-text {  
  user-select: auto;  
  /* Override for specific sections */  
}
```

Result: Mostly non-selectable site, with select content unlocked. **Not recommended** - frustrates users.

5.10 Example 10: Form Labels

```
label {  
  user-select: none;  
  /* Label text won't highlight when clicking */  
  display: block;  
  margin-bottom: 8px;  
  font-weight: 500;  
  cursor: pointer;  
}
```

```
input,
textarea {
  user-select: text;
  /* Form inputs remain selectable */
}
```

Result: Clean form without accidental label selection.

Chapter 6

Common Mistakes

6.1 Mistake 1: Using none on Article Text

Very Wrong:

```
article {  
  user-select: none;  
  /* NO! Users can't read or copy content */  
}
```

Result: Frustrates readers, hurts SEO and accessibility.

Correct:

```
article {  
  user-select: auto;  
  /* Let users read and copy */  
}  
  
article .publish-date {  
  user-select: none;  
  /* Only disable for UI elements */  
}
```

6.2 Mistake 2: Thinking user-select Prevents Copying

Wrong assumption:

```
.protected {  
  user-select: none;  
  /* This does NOT protect content! */  
}
```

```
/*  
Reality: Users can still:  
- View source (Ctrl+U)  
- Copy from browser cache  
- Use developer tools  
- Screenshot the page  
- Use screen readers
```

```
*/
```

Truth: User-select is for UX, not security. Don't rely on it to protect content.

6.3 Mistake 3: Over-Protecting UI

Excessive:

```
* {  
  user-select: none;  
  /* Everything disabled - terrible UX */  
}
```

Better:

```
button, label, nav {  
  user-select: none;  
  /* Only disable specific UI elements */  
}  
  
/* Everything else is selectable by default */
```

6.4 Mistake 4: Forgetting Vendor Prefixes

Limited browser support:

```
.element {  
  user-select: none;  
  /* Doesn't work in older browsers */  
}
```

Better:

```
.element {  
  -webkit-user-select: none;  
  -moz-user-select: none;  
  -ms-user-select: none;  
  user-select: none;  
  /* Works in all browsers */  
}
```

6.5 Mistake 5: Using none on Accessible Content

Bad for accessibility:

```
.help-text {  
  user-select: none;  
  /* Screen reader users need this text */  
}
```

Remember: user-select doesn't affect screen readers. Use it for visual UI only, not essential content.

Chapter 7

Accessibility Considerations

7.1 Screen Readers are Unaffected

Important: `user-select` only affects mouse/keyboard selection:

- Screen reader users can still access text
- Keyboard navigation unaffected
- Copy-to-clipboard shortcuts may work differently
- Browser search (Ctrl+F) still finds text

7.2 Best Practices for Accessibility

1. Only disable selection for decorative UI elements
2. Never disable for article/blog content
3. Never disable for critical information
4. Ensure essential text is readable by all users
5. Test with assistive technology
6. Document why you disabled selection

Chapter 8

Browser Support

8.1 Current Support

User-select is well-supported in all modern browsers:

- Chrome 43+
- Firefox 69+ (earlier with `-moz-` prefix)
- Safari 3+ (with `-webkit-` prefix)
- Edge 79+
- Opera 30+
- iOS Safari 3+
- Chrome Mobile 43+
- Samsung Internet 4+
- IE 10+ (with `-ms-` prefix, limited support)

8.2 Always Use Vendor Prefixes

```
.element {  
  -webkit-user-select: none;  
  -moz-user-select: none;  
  -ms-user-select: none;  
  user-select: none;  
}
```

Chapter 9

Related CSS Properties

9.1 Complementary Properties

9.1.1 cursor

```
.button {  
  user-select: none;  
  cursor: pointer;  
  /* Shows it's clickable */  
}
```

9.1.2 pointer-events

```
.tooltip {  
  user-select: none;  
  pointer-events: none;  
  /* Won't interfere with interactions */  
}
```

9.1.3 user-drag

```
.draggable {  
  user-select: none;  
  user-drag: element;  
  /* Combines with dragging */  
}
```

Chapter 10

Quick Reference

10.1 User-Select Values Summary

Value	Behavior	Use Case
auto	Select normally	Content (default)
none	Cannot select	UI elements
text	Select normally	Content (explicit)
all	Select entire block	Code blocks
contain	Select within bounds	Bounded regions

10.2 Complete CSS Template

```
/* Protect UI elements */
button, nav, label, .button-like {
  -webkit-user-select: none;
  -moz-user-select: none;
  -ms-user-select: none;
  user-select: none;
}

/* Allow article content */
article, .content, .text {
  -webkit-user-select: text;
  -moz-user-select: text;
  -ms-user-select: text;
  user-select: text;
}

/* Code blocks - select all */
code, pre, .code-block {
  -webkit-user-select: all;
  -moz-user-select: all;
  -ms-user-select: all;
  user-select: all;
}
```

10.3 Decision Flowchart

1. Is this text UI (button, label, nav)?
 - YES → Use **none**
 - NO → Continue
2. Is this article or main content?
 - YES → Use **auto** (or omit)
 - NO → Continue
3. Is this code meant to be copied?
 - YES → Use **all**
 - NO → Use **auto**

10.4 Common Patterns

- **Navigation:** none
- **Buttons:** none
- **Labels:** none
- **Article text:** auto
- **Code blocks:** all
- **Icons:** none
- **Draggable items:** none
- **Form labels:** none

10.5 Pro Tips

1. Default to allowing selection (better UX)
2. Only disable for UI elements
3. Always include vendor prefixes
4. Test with keyboard and mouse
5. Don't rely on user-select for security
6. Consider accessibility implications
7. Document your choices in code comments

10.6 When NOT to Use `user-select: none`

- Body text and articles (allow copying)
- Blog posts and documentation
- Legal text and terms
- Educational content
- Technical documentation
- Help text
- Error messages
- Important information

10.7 When to Use `user-select: none`

- Button labels
- Navigation links
- Form labels
- Icon labels
- UI controls
- Menu items
- Decorative elements
- Drag-and-drop handles